

References

F. Devernay and O. Faugeras. SPIE Conference on investigative and trial image processing. SanDiego, CA, 1995. Automatic calibration and removal of distortion from scenes of structured environments.

H. Farid and A.C. Popescu. Journal of the Optical Society of America, 2001. Blind removal of Lens Distortion

R. Swaminatha and S.K. Nayer. IEEE Conference on computer Vision and pattern recognition, pp 413, 1999. Non-metric calibration of wide angle lenses and poly-cameras

G. Taubin. Lecture notes EE-148, 3D Photography, Caltech, 2001. Camera model for triangulation

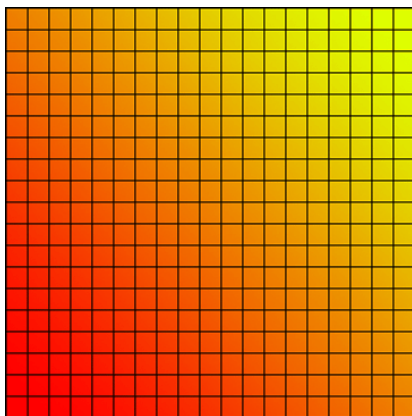
Non-linear Lens Distortion

With an example using [OpenGL](#) ([lens.c](#), [lens.h](#))

Written by [Paul Bourke](#)

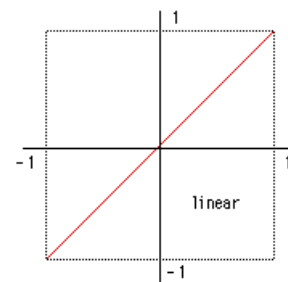
August 2000

The following illustrates a method of forming arbitrary non linear lens distortions. It is straightforward to apply this technique to any image or 3D rendering, examples will be given here for a few mathematical distortion functions but the approach can use any function, the effects are limited only by your imagination. At the end an [OpenGL](#) application is given that implements the technique in real-time (given suitable OpenGL hardware and texture memory).



This is the sample input image that will be used to illustrate a couple of different distortion functions.

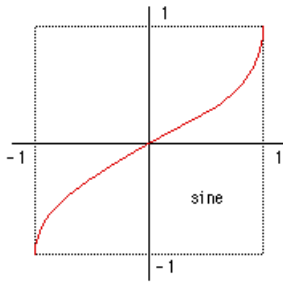
Consider the linear function below:



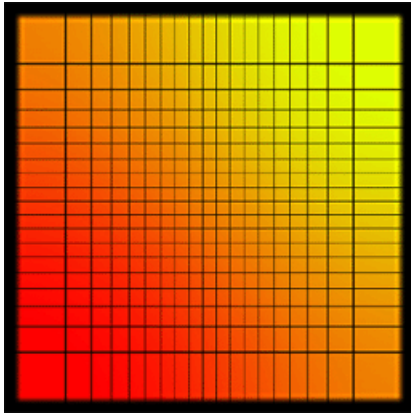
The horizontal axes is the coordinate in the new image, the vertical axis is the coordinate in the original image. To find the corresponding pixel in the new image one locates the value on the horizontal axis and moves up to the red line and reads off the value on the vertical axis. The linear function above would result in an output image that looks the same as the input image.

sine

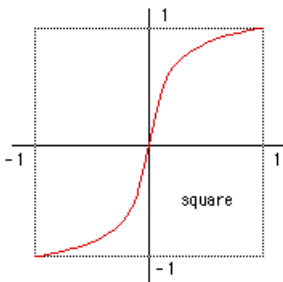
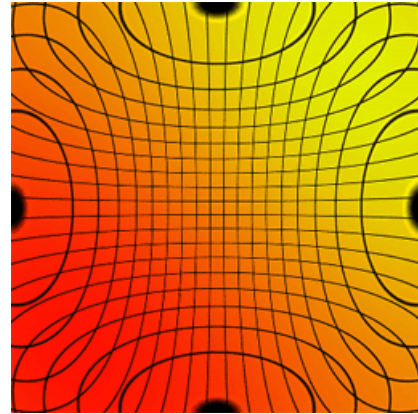
A more interesting example is based upon a sine curve. You should be able to convince yourself that this function will stretch values near +1 and -1 while compressing values near the origin. An important requirement for these distortion functions is they need to be strictly one-to-one, that is, there is a unique vertical value for each horizontal value (and visa-versa). If image flipping is disallowed then this implies the distortion function is always increasing as one moves from left to right along the horizontal axis.



There are two ways of applying this function to an image, the first shown on the left in each example below applies the function to the horizontal and vertical coordinates of the image. The example on the right applies the function to the radius from the center of the image, the angle is undistorted.



square



There are a number of ways the image coordinates are mapped onto the function range. The approach used here was to scale and translate the image coordinates so that 0 is in the center of the image and the bounds of the image range from -1 to +1. This is done twice, one to map the output image coordinates to the -1 to +1 range, the function is then applied, and then the inverse transformation maps the -1 to +1 range onto the range in the input image.

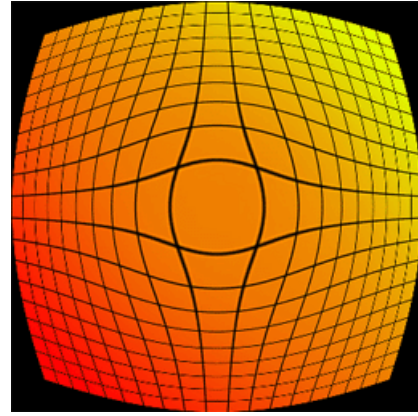
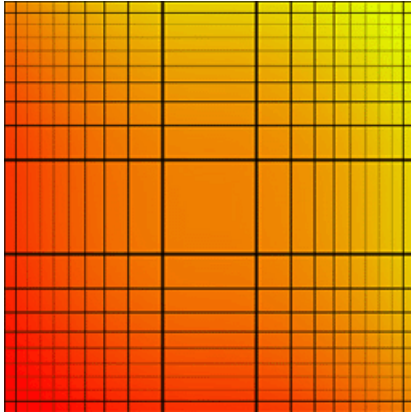
So if i_{out} and j_{out} are the coordinates of the output image, and w_{out} and h_{out} the output image dimensions, then the mapping onto the -1 to +1 range is

$$x_{out} = i_{out} / (w_{out}/2) - 1, \text{ and } y_{out} = j_{out} / (h_{out}/2) - 1$$

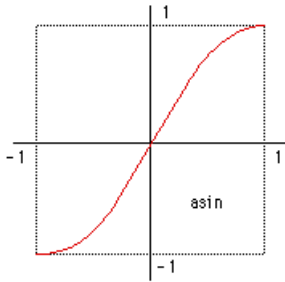
Applying the function to x_{in} and y_{in} gives x_{new} and y_{new} . The inverse mapping from the x_{new} and y_{new} gives i_{in} and j_{in} (the index in the input image with a width of w_{in} and h_{in}) is just

$$i_{in} = (x_{new} + 1) * (w_{in}/2), \text{ and } j_{in} = (y_{new} + 1) * (h_{in}/2)$$

Given i_{in} and j_{in} the colour in the input image can be applied to pixel i_{out} , j_{out} in the output image.



asin



Applying the function to polar coordinates is only slightly different. The radius and angle of a pixel is computed based up x_{out} and y_{out} . The radius lies between 0 and 1 so the positive half of the function is used to transform it. The pixel coordinates in the input image are calculated using the new radius and the unchanged angle.

Using the conventions above:

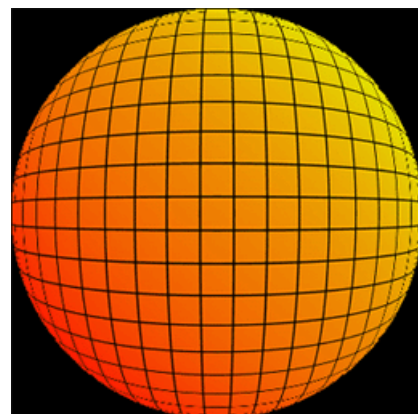
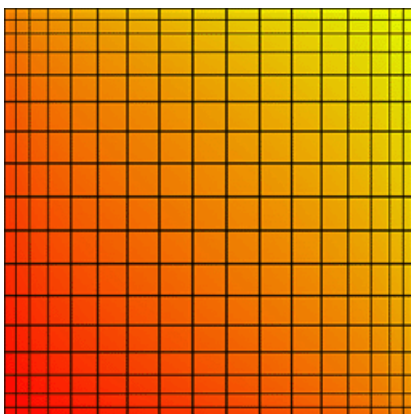
$$r_{out} = \sqrt{x_{out}^2 + y_{out}^2}, \text{ and } angle_{out} = \text{atan2}(y_{out}, x_{out})$$

The transformation is applied to r_{out} to give r_{new} , x_{new} and y_{new} is calculated as

$$x_{in} = r_{new} \cos(angle_{out}), \text{ and } y_{in} = r_{new} \sin(angle_{out})$$

i_{in} and j_{in} are calculated as before from x_{in} and y_{in} .

Note that in both cases (distorting the Cartesian coordinates or polar coordinates) it is possible for there to be an un-mappable region, that is, coordinates in the new image which when distorted lie outside the bounds of the input image.



Notes on resolution

Some parts of the image are compressed and other parts inflated, the inflated regions need a higher input image resolution in order to be represented without aliasing effects. The above transformations cope with the input and output images being different sizes, normally the input image needs to be much larger than the output image. To

Image warping / distortion

Written by [Paul Bourke](#)

December 2002

This documents, primarily with examples, an image warping application that was originally developed to test the simulation of various lens types. There are a few key ideas with image transformations such as this.

- The transformation that one needs is not the function that maps from the source image to the destination but the reverse. What is required is to find the source pixel associated with each pixel in the destination image. In general the image plane is considered to be a real valued function rather than a discrete pixel plane, this is particularly so when antialiasing is implemented.
- The image coordinates are transformed from pixels (i,j) ranging from 0 to the width-1 and height-1 to normalised coordinates (x,y) ranging from -1 to 1, this is irrespective of the image proportions and it applies to both the source and destination image coordinates.

$$x = 2 i / \text{width} - 1$$

$$y = 2 j / \text{height} - 1$$

- A key ingredient for image quality is antialiasing. In the code given here, supersampling is used. That is, each pixel in the destination image is sampled a number of times, the resulting estimate from the source image are averaged (box filter is used here) to give a final estimate in the destination image.
- Some transformation act upon the cartesian coordinates while other are radial (polar coordinates) in nature. The r,phi coordinates are as follows.

$$r = \sqrt{x^2 + y^2}$$

$$\phi = \text{atan2}(y,x)$$

After the radial based warp the reverse transformation back to cartesian normalised coordinates is:

$$x = r \cos(\phi)$$

$$y = r \sin(\phi)$$

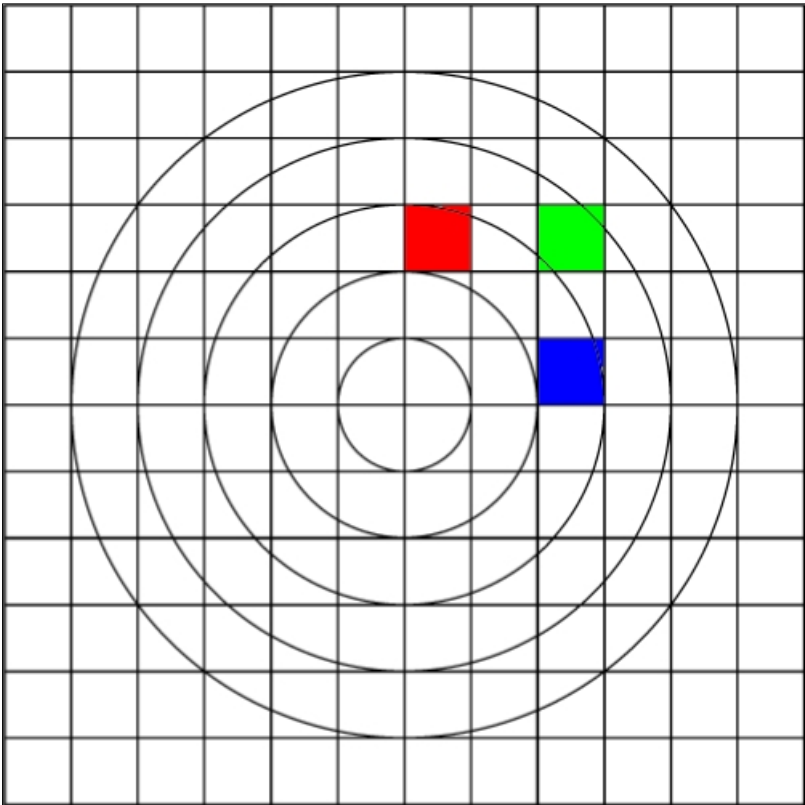
This application ([imagewarp.c](#)) is written as a UNIX utility that reads a TGA file (24 or 32 bit), performs the warping as specified by command line arguments and writes the resulting image as a TGA file to standard output (to be redirected to a file say).

```
imagewarp tgafilename [options]
Options:
-a n    set antialias level (Default: 1)
-w n    width of the output image (Default: 500)
-h n    height of the output image (Default: width)
-m n    mapping type (Default: 0)
-p1 n   first parameter for mapping (Default: 0)
-p2 n   second parameter for mapping (Default: 0)
-p3 n   third parameter for mapping (Default: 0)
-s n    scale factor (Default: 1)
-v      debug/verbose mode (Default: off)
```

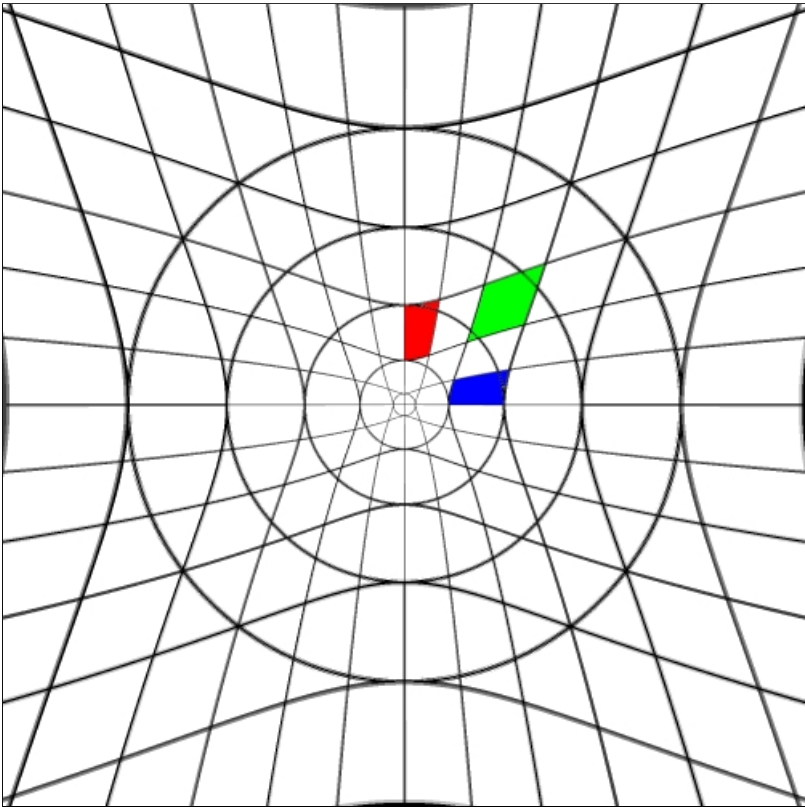
The following lists each of the image warping types supported, other warping function can readily be added.

Source image

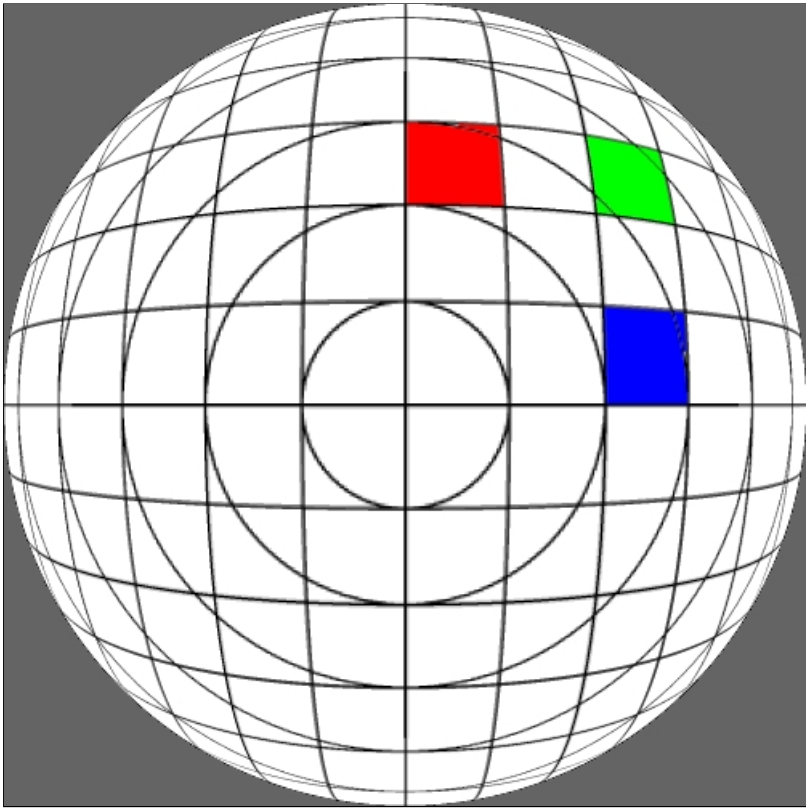
This is the test image:
[test.tga](#) that is transformed
into each of the images
below.



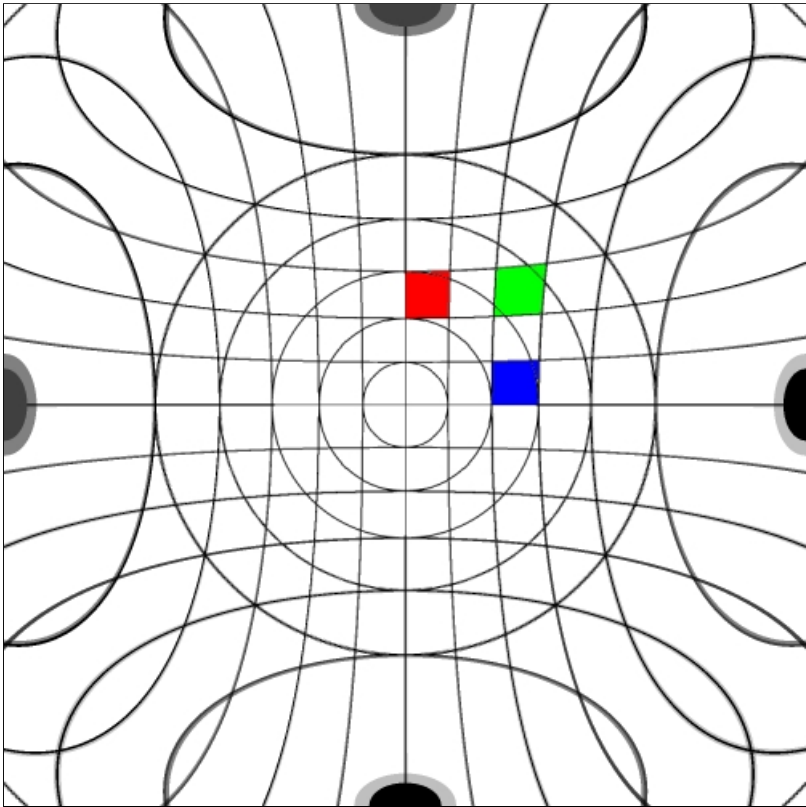
$r \rightarrow \sqrt{r}$



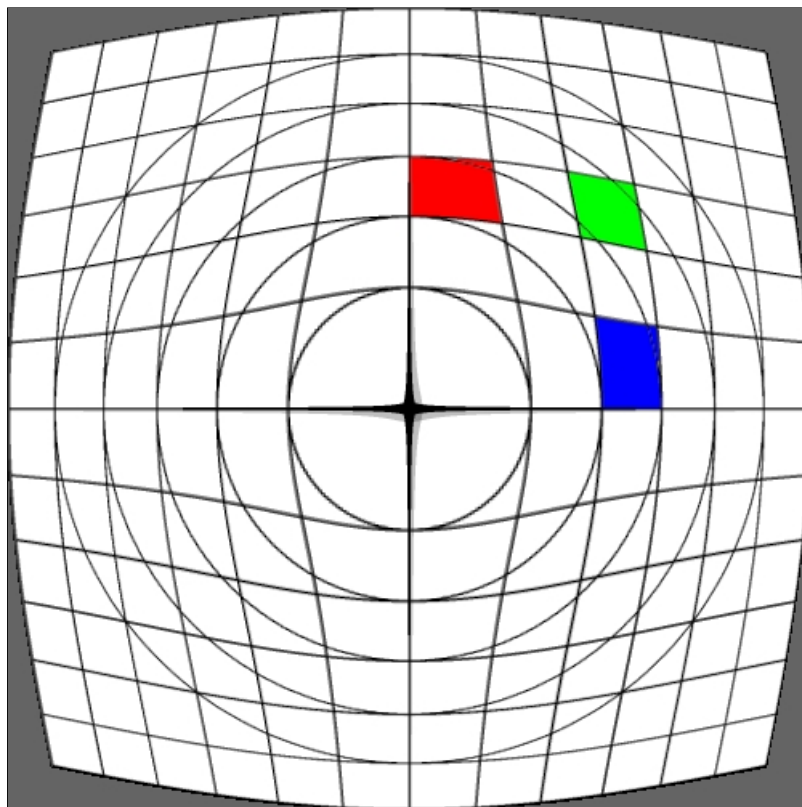
$r \rightarrow 2 \arcsin(r) / \pi$



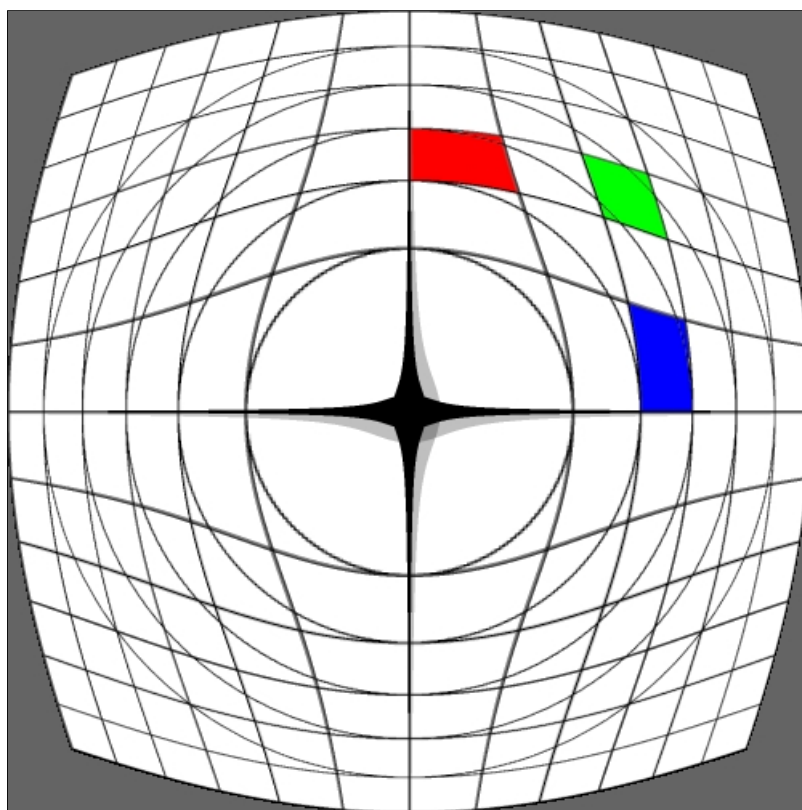
$r \rightarrow \sin(\pi r / 2)$



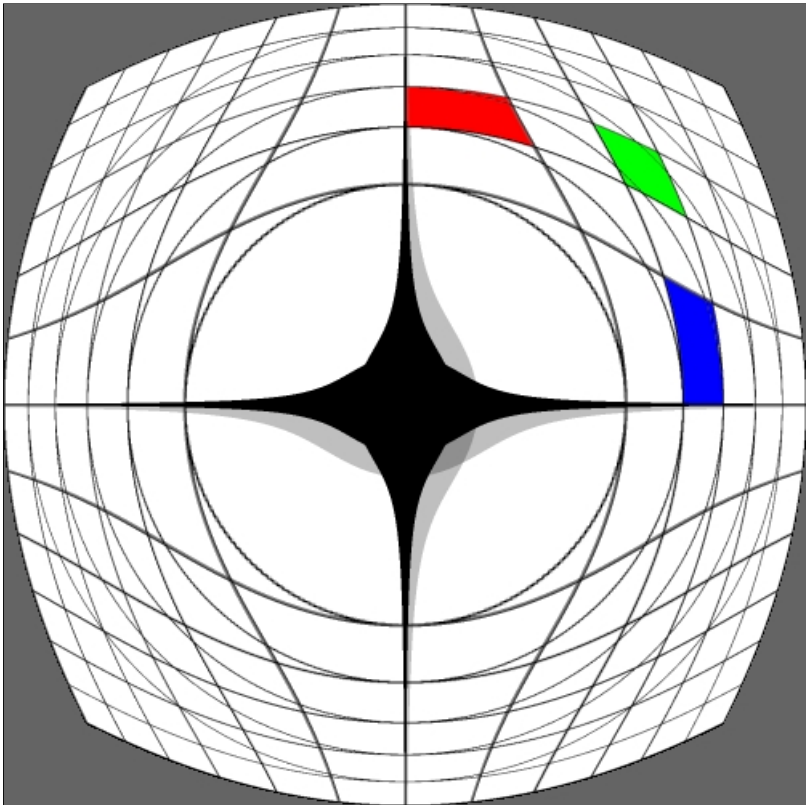
$$r \rightarrow r^p$$
$$p = 1.5$$



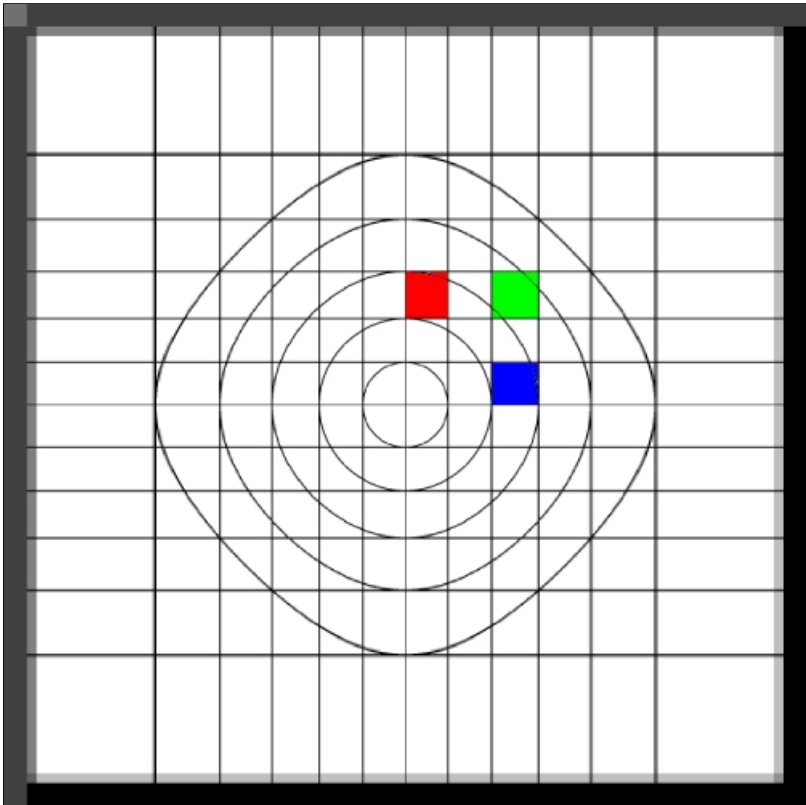
$$r \rightarrow r^p$$
$$p = 2$$



$r \rightarrow r^p$
 $p = 3$

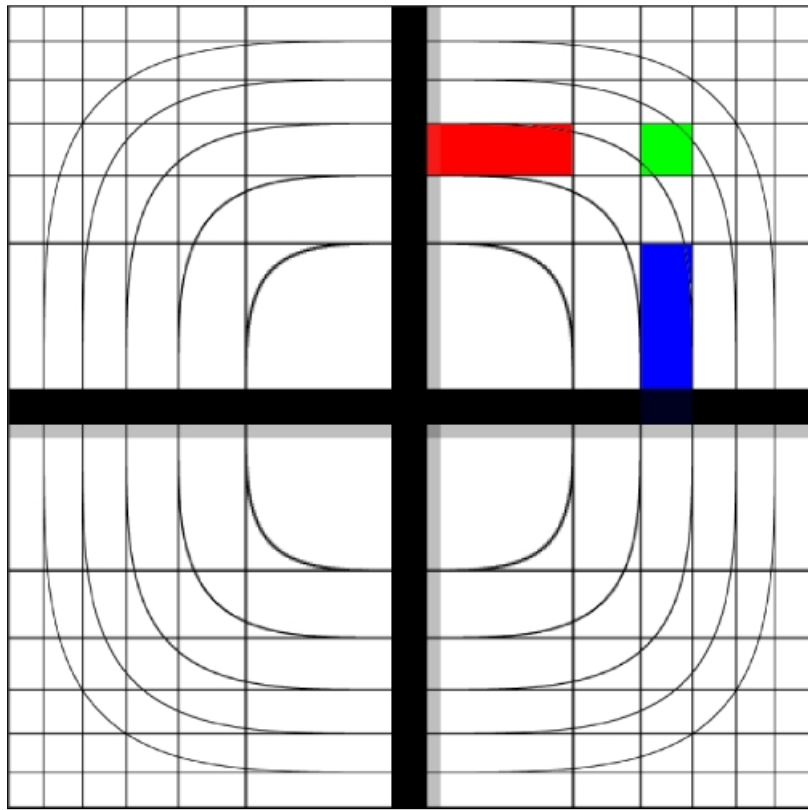


$x \rightarrow \sin(\pi x / 2)$
 $y \rightarrow \sin(\pi x / 2)$



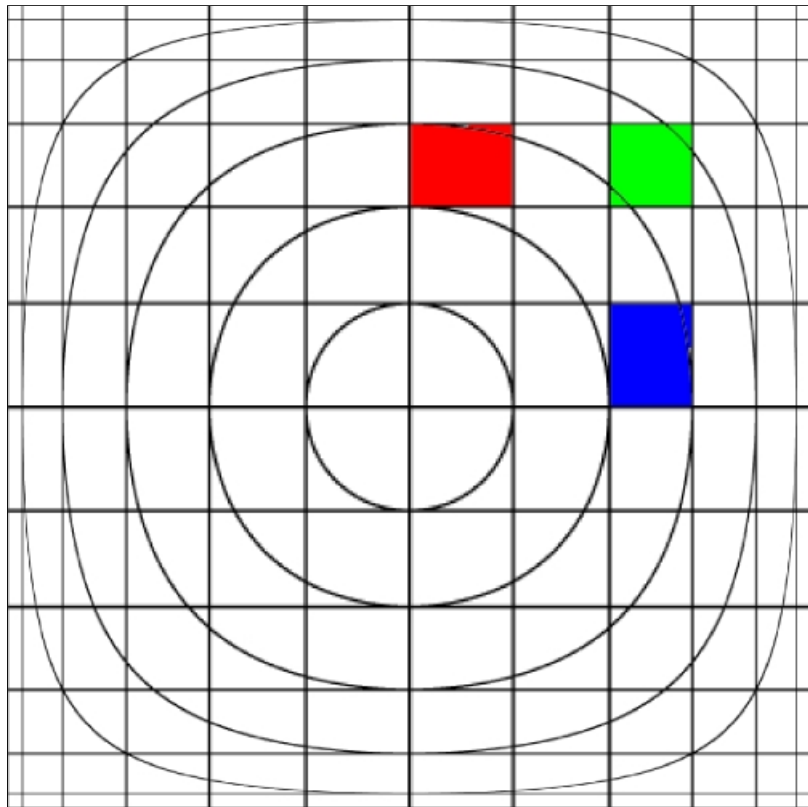
$$x \rightarrow \text{sign}(x) x^2$$

$$y \rightarrow \text{sign}(y) y^2$$

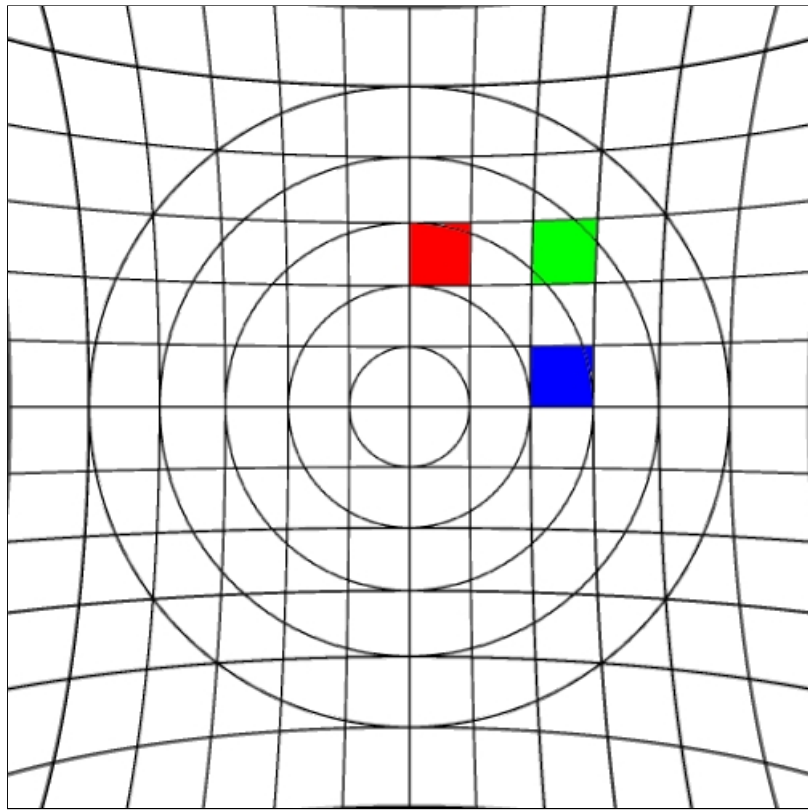


$$x \rightarrow 2 \arcsin(x) / \pi$$

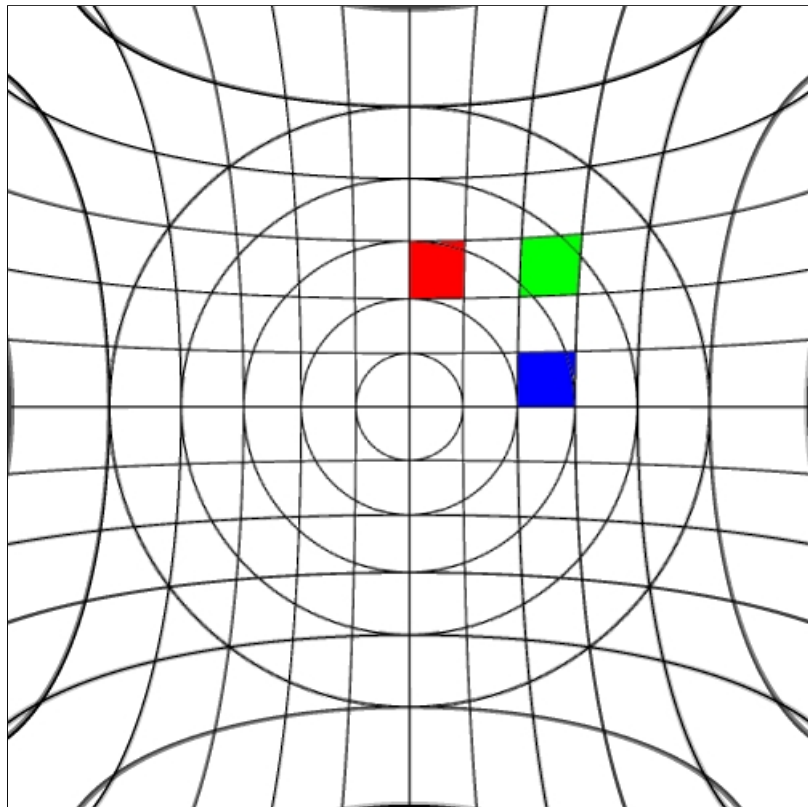
$$y \rightarrow 2 \arcsin(y) / \pi$$



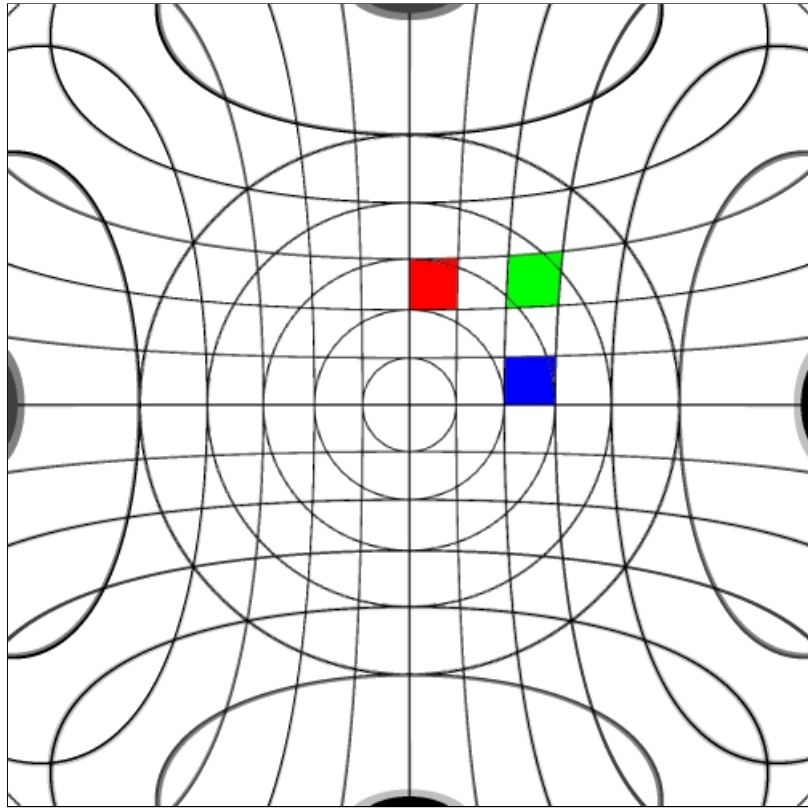
```
x -> x (1 - p r) / (1 - p)  
y -> y (1 - p.r) / (1 - p)  
p = 0.1
```



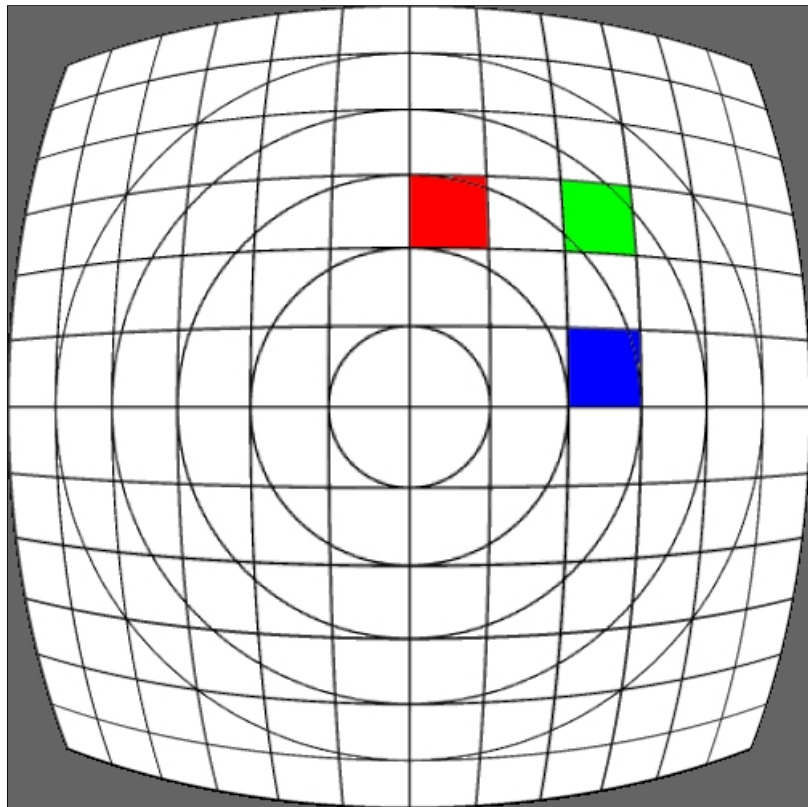
```
x -> x (1 - p r) / (1 - p)  
y -> y (1 - p.r) / (1 - p)  
p = 0.2
```



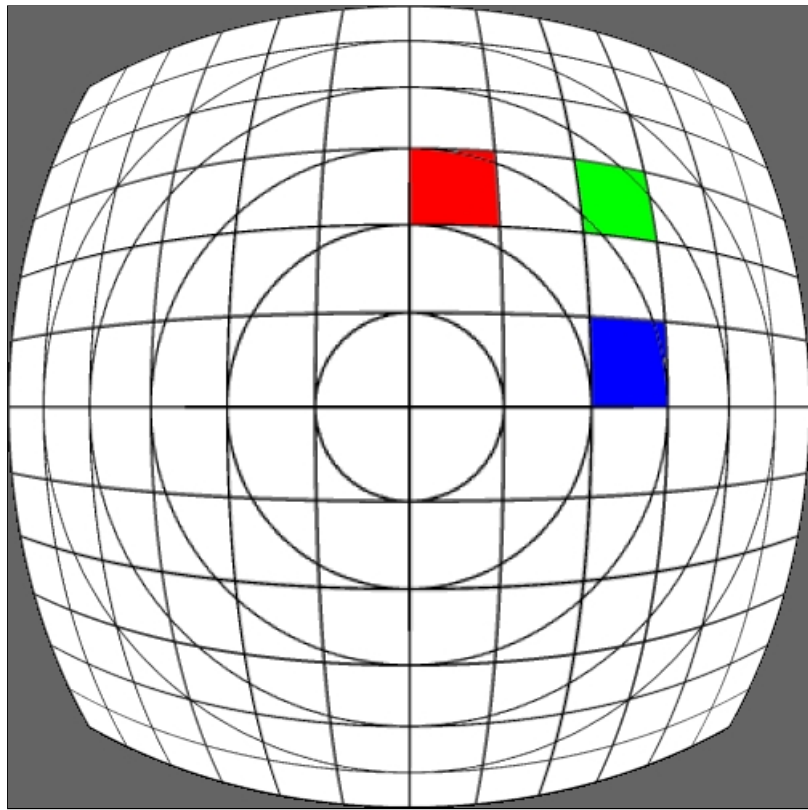
$x \rightarrow x(1 - p r) / (1 - p)$
 $y \rightarrow y(1 - p r) / (1 - p)$
 $p = 0.3$



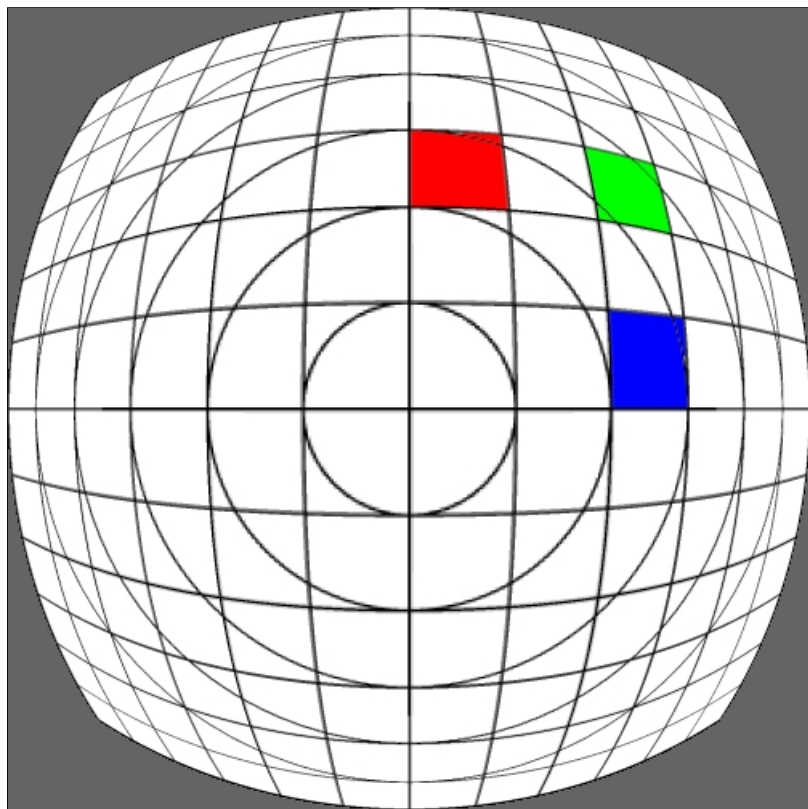
Method by H. Farid and
A.C. Popescu for modest
lens with good fit.
 $p = 0.2$



Method by H. Farid and
A.C. Popescu for modest
lens with good fit.
 $p = 0.4$



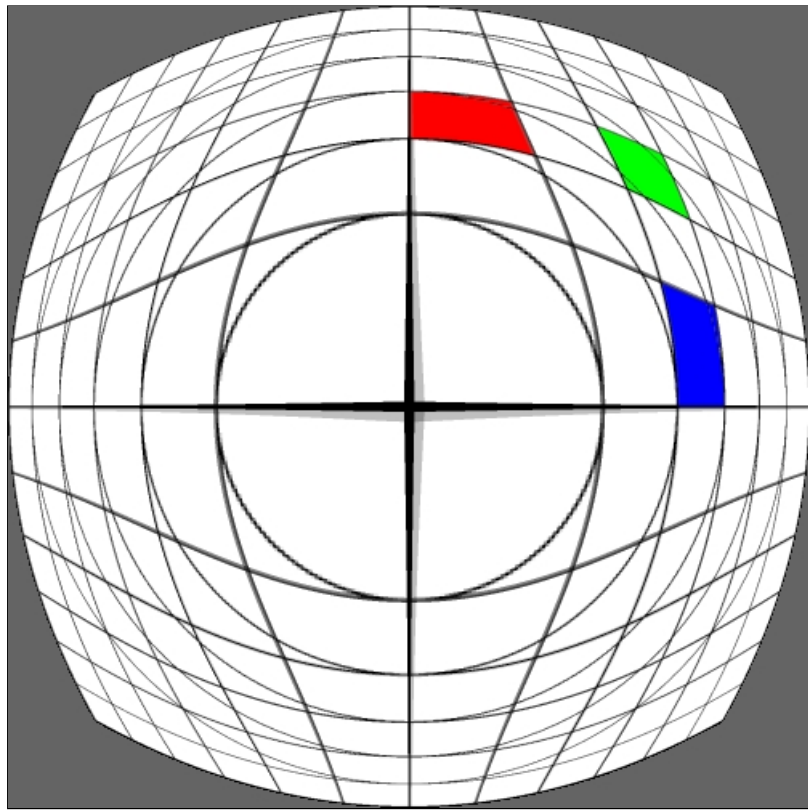
Method by H. Farid and
A.C. Popescu for modest
lens with good fit.
 $p = 0.6$



$$r \rightarrow p_1 10^r p_2 - p_1$$

$$p_1 = 0.049$$

$$p_2 = 1.280$$

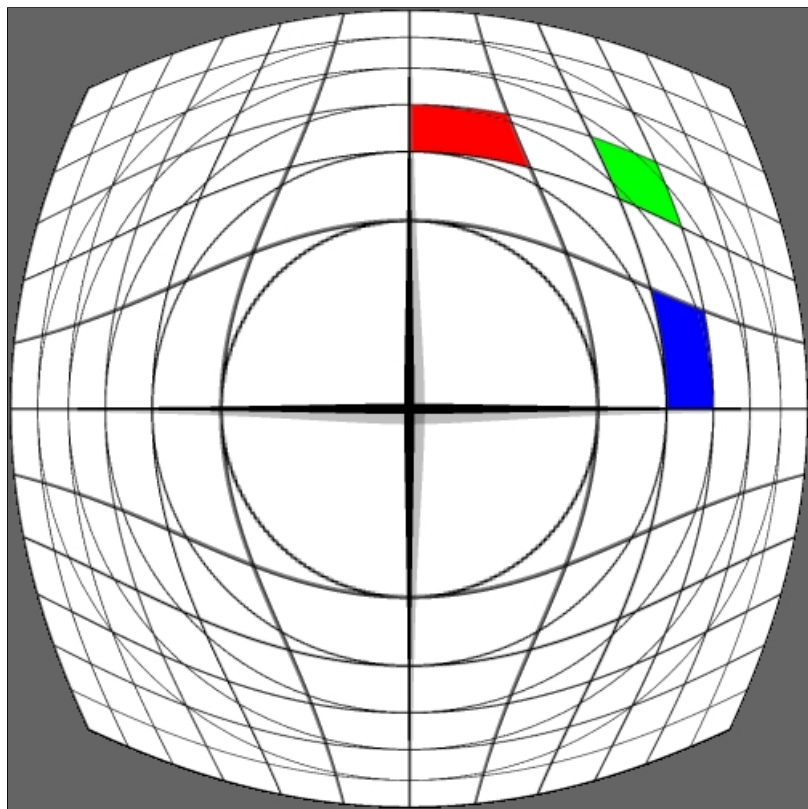


$$r \rightarrow p_1 r^3 + p_2 r^2 + p_3 r$$

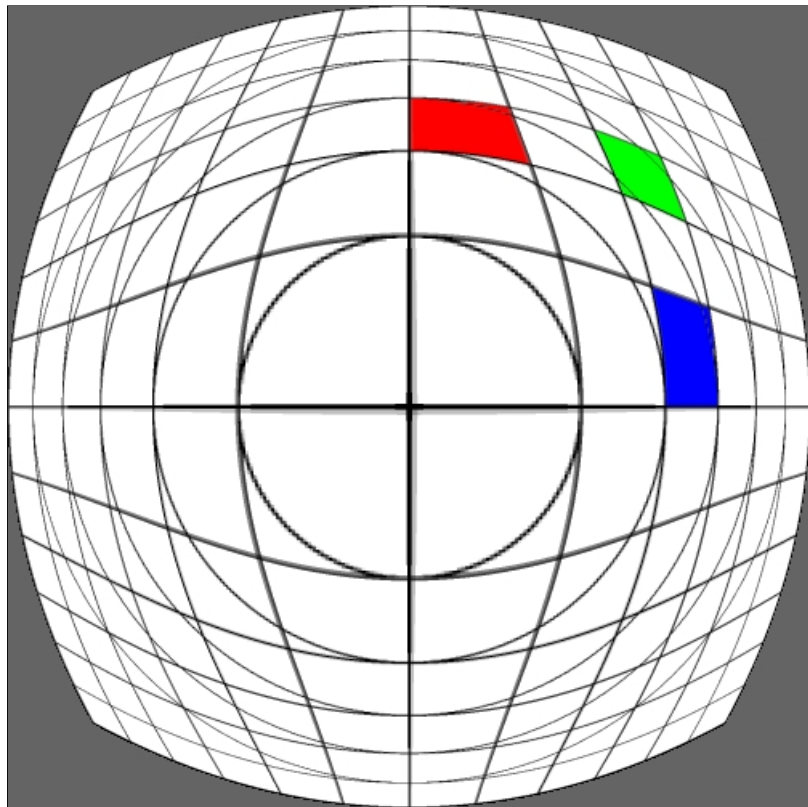
$$p_1 = 0.819$$

$$p_2 = 0.046$$

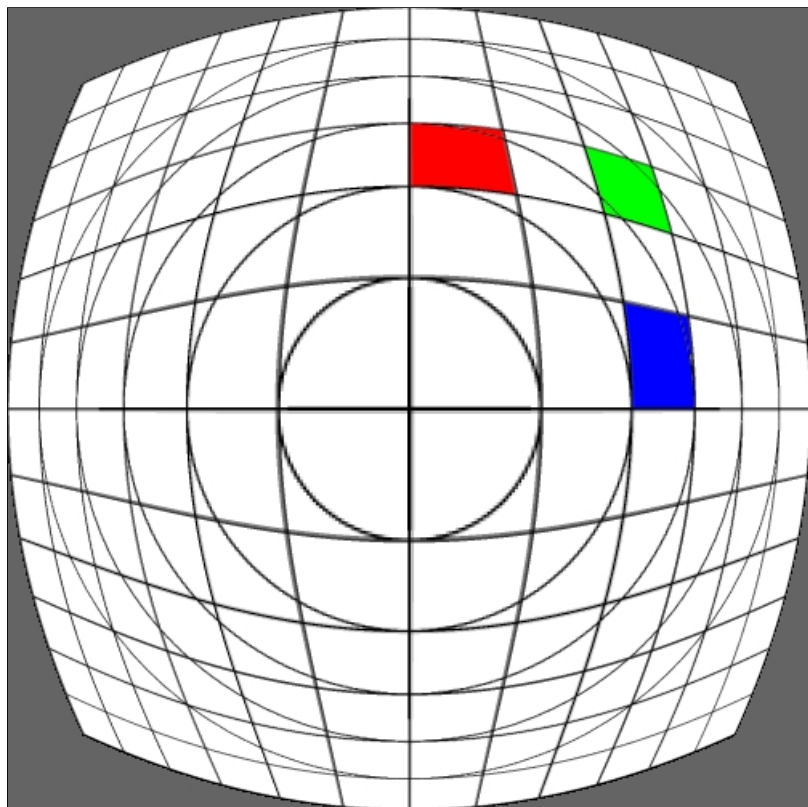
$$p_3 = 0.152$$



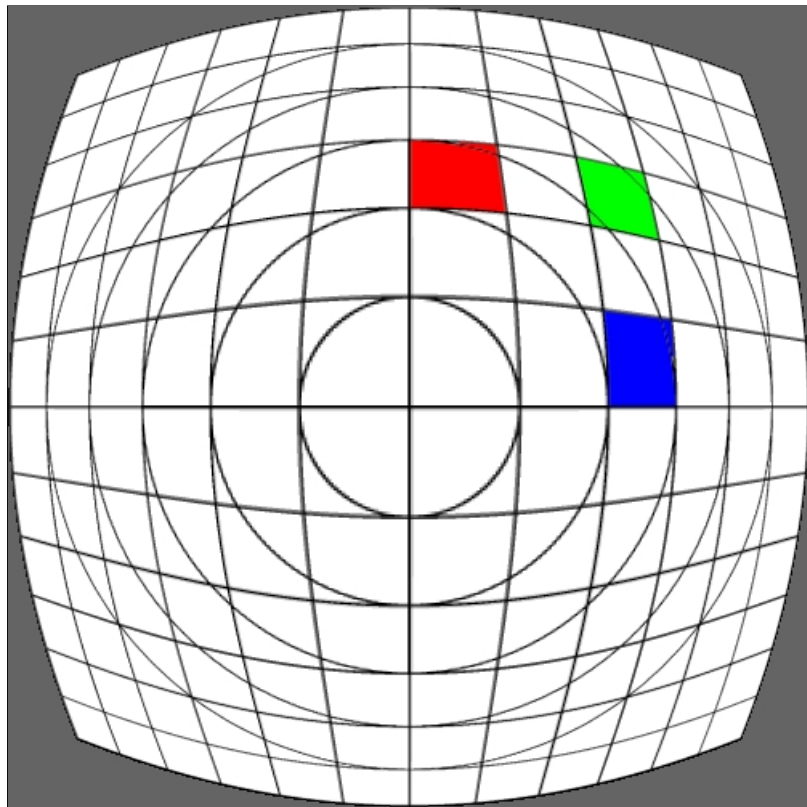
Janez Pers, Stanislav
Kovacic An Alternative
Model for Radial Distortion
in Wide-Angle Lenses
(Single parameter model)
 $p = 0.1$



Janez Pers, Stanislav
Kovacic An Alternative
Model for Radial Distortion
in Wide-Angle Lenses
(Single parameter model)
 $p = 0.2$



Janez Pers, Stanislav
Kovacic An Alternative
Model for Radial Distortion
in Wide-Angle Lenses
(Single parameter model)
 $p = 0.3$



Rotation
 $\phi = \phi + p$
 $p = 0.5$

